Assignment 2: A Line-by-Line Explanation

This guide explains the completed code in `Problem2_soln.ipynb` and the existing solution in `Problem1_HMM_question.ipynb`. The Problem 2 starter notebook, its classes, its sample network, and its supplied checks are preserved. Problem 1 is explained without modifying it.

Each code listing reproduces the notebook source. Line numbers restart for each listing and include blank lines. The explanations cover every nonblank line; blank lines only separate steps. Indentation determines which statements belong to a loop or condition. A variable assigned inside a loop is a local working value.

Contents

1 Problem 2: Variable elimination

1.1 What the data structures mean

A `Variable` has a name and a list of possible values called its domain. For example, a Boolean variable may have domain `[True, False]`. The output distribution must follow this order; it must not assume False comes first. A `Factor` is a table of numbers over an ordered list of variables called its scope. If the scope is `[X, Y]`, the key `(0, 1)` means $X = 0, Y = 1$. If the scope is `[Y, X]`, the same key means $Y = 0, X = 1$. Scope order therefore matters. A scope with no variables represents a constant and has one key, `()`. A factor need not sum to one: it may be an intermediate product or sum. A `BN` stores the network variables and its conditional probability factors.

The supplied `get_scope()`, `variables()`, and `factors()` methods return new lists. However, the objects inside these lists are still the same objects. The solution reads original factors and builds new ones for every operation that changes a table. It never edits the network's original tables. The class definitions and sample network are given code, not functions to implement.

1.2 The overall calculation

For a query variable Q, evidence e, and all remaining hidden variables H,

$$P(Q = q \mid e) = \frac{w(q)}{\sum_{q'} w(q')}, \qquad w(q) = \sum_{H} \prod_{i} f_i|_{Q=q,E=e}.$$

We first fix evidence values in the factor tables. For each hidden variable we multiply all factors containing it, then sum it out. Finally we multiply the remaining factors and normalize the query weights. The implementation uses the network's given variable order: it does not search for a faster elimination order.

1.3 A small example before the code

Suppose a factor over `[X, Y]` has rows $(0,0) : 0.1$, $(0,1) : 0.9$, $(1,0) : 0.2$, and $(1,1) : 0.8$. Restricting to $Y = 1$ keeps only $X = 0 : 0.9$ and $X = 1 : 0.8$. Summing out Y instead gives $X = 0 : 1.0$ and $X = 1 : 1.0$. If another factor gives $Y = 0 : 0.3$ and $Y = 1 : 0.7$, multiplying it with the original factor gives $(0,0) : 0.03$, $(0,1) : 0.63$, $(1,0) : 0.06$, and $(1,1) : 0.56$.

1.4 normalize

```
1 def normalize(nums):
2     """Return a new list proportional to nums that sums to one."""
3     total = sum(nums)
4     if total == 0:
5         raise ValueError("Cannot normalize numbers whose sum is zero.")
6     probabilities = []
7     for number in nums:
8         probabilities.append(number / total)
9     return probabilities
```

Line 1. Define a function that converts a list of weights into probabilities.

Line 2. Document that the output is a new list, so the input is preserved.

Line 3. Add all input weights. This is the common denominator for normalization.

Line 4. Check whether normalization would require division by zero.

Line 5. Report that a zero total cannot be turned into a proportional distribution summing to one.

Line 6. Create a separate list for the output probabilities.

Line 7. Visit each input weight in its original order.

Line 8. Divide the current weight by the total and append the resulting probability.

Line 9. Return the new list. For example, weights [1, 3] become [0.25, 0.75].

The implementation intentionally raises an error for a zero total, including an empty list. Replacing zero by a small epsilon would not produce a distribution summing to one. The assignment provides numerical weights; the factors used in probability calculations have nonnegative entries.

1.5 restrict_factor

```
1  def restrict_factor(f, var, value):
2      """Return a new factor with var fixed to value and removed from its scope."""
3      position = f.scope.index(var)
4      if value not in var.domain():
5          raise ValueError("Evidence value is outside the variable's domain.")
6      new_scope = f.get_scope()
7      new_scope.remove(var)
8      result = Factor("restricted", new_scope)
9      for assignment, probability in f.values.items():
10         if assignment[position] == value:
11             new_assignment = assignment[:position] + assignment[position + 1:]
12             result.set_value(new_assignment, probability)
13     return result
```

Line 1. Define restriction: fix one variable to an observed value.

Line 2. Describe the operation and its new-factor guarantee.

Line 3. Find the variable's column in each assignment tuple. An absent variable raises ValueError.

Line 4. Check that the requested observation is a permitted domain value.

Line 5. Reject an invalid observation instead of silently creating an unusable factor.

Line 6. Get a copy of the scope so that removing a variable cannot change the input factor.

Line 7. Remove the fixed or eliminated variable from the output scope.

Line 8. Create an empty output factor with the smaller scope.

Line 9. Read each assignment tuple and its stored numerical value without changing the input table.

Line 10. Keep only rows whose value in the evidence column matches the observation.

Line 11. Join the tuple entries before and after the removed column. The original tuple is unchanged.

Line 12. Copy the matching row's value into the output under its shorter assignment.

Line 13. Return the newly built output, leaving the input factors unchanged.

1.6 sum_out_variable

```
def sum_out_variable(f, var):
    """Return a new factor with var removed by summing over its values."""
    position = f.scope.index(var)
    new_scope = f.get_scope()
    new_scope.remove(var)
    result = Factor("summed", new_scope)
    for assignment, probability in f.values.items():
        new_assignment = assignment[:position] + assignment[position + 1:]
        previous_total = result.values.get(new_assignment, 0)
        result.set_value(new_assignment, previous_total + probability)
    return result
```

Line 1. Define marginalization: remove a variable by adding over its possible values.

Line 2. Describe summation rather than choosing one evidence value.

Line 3. Find the variable's column in each assignment tuple. An absent variable raises ValueError.

Line 4. Get a copy of the scope so that removing a variable cannot change the input factor.

Line 5. Remove the fixed or eliminated variable from the output scope.

Line 6. Create an empty factor over all the variables that remain.

Line 7. Read each assignment tuple and its stored numerical value without changing the input table.

Line 8. Join the tuple entries before and after the removed column. The original tuple is unchanged.

Line 9. Read the accumulated sum for the shortened row, or use zero if this is its first contribution.

Line 10. Add this input row to the running sum of all rows that agree on the remaining variables.

Line 11. Return the newly built output, leaving the input factors unchanged.

For example, removing the first column of `(True, False)` leaves `(False,)`. Input rows that produce the same shortened tuple contribute to the same sum. Removing the only variable produces a constant factor with key `()`. The code expects complete factor tables, as in the supplied classes and sample network.

1.7 multiply_factors

```
1  def multiply_factors(factors):
2      """Return a new factor containing the product of all input factors."""
3      scope = []
4      for factor in factors:
5          for var in factor.scope:
6              if var not in scope:
7                  scope.append(var)
8      result = Factor("product", scope)
9      domains = []
10     for var in scope:
11         domains.append(var.domain())
12     for assignment in product(*domains):
13         probability = 1.0
14         for factor in factors:
15             factor_assignment = []
16             for var in factor.scope:
17                 position = scope.index(var)
18                 factor_assignment.append(assignment[position])
19             probability *= factor.get_value(factor_assignment)
20         result.set_value(assignment, probability)
21     return result
```

Line 1. Define a function that combines any number of factor tables by multiplication.

Line 2. Document the product operation and preservation of the inputs.

Line 3. Start an ordered list of all variables that appear in the input factors.

Line 4. Visit each factor in the current list.

Line 5. Visit the variables in this factor's own column order.

Line 6. Check whether this variable has already been included in the combined scope.

Line 7. Include this variable once, preserving the order of first appearance.

Line 8. Create a new factor whose scope is the union of the input scopes.

Line 9. Prepare a list holding the possible values for each combined-scope variable.

Line 10. Visit the variables in the combined scope order.

Line 11. Append this variable's domain so that assignments follow the same order as the scope.

Line 12. Enumerate every combination of domain values. The star passes each domain separately to product.

Line 13. Start a running product at one, the identity for multiplication.

Line 14. Visit each factor in the current list.

Line 15. Prepare the values needed to look up one input factor's table row.

Line 16. Visit the variables in this factor's own column order.

Line 17. Locate this input-factor variable in the combined assignment.

Line 18. Copy its value into the input factor's own scope order; different factors may order columns differently.

Line 19. Look up the input factor's value and multiply it into the running product.

Line 20. Store the completed product for this combined assignment.

Line 21. Return the newly built output, leaving the input factors unchanged.

With no input factors, the product is the constant one. Python's `product(*[])` produces the single empty assignment `()`, so the same code returns the factor $\{() : 1.0\}$. Constant input factors are included in every product, even though they have no variables.

1.8 VE

```
def VE(bayes_net, var_query, varlist_evidence):
    """Return the query distribution in domain order without changing input factors.
    """
    if var_query not in bayes_net.variables():
        raise ValueError("The query variable must belong to the network.")
    factors = bayes_net.factors()
    for evidence in varlist_evidence:
        if evidence not in bayes_net.variables():
            raise ValueError("Evidence variables must belong to the network.")
        value = evidence.get_evidence()
        if value not in evidence.domain():
            raise ValueError("Set an evidence value before calling VE.")
        restricted_factors = []
        for factor in factors:
            if evidence in factor.scope:
                factor = restrict_factor(factor, evidence, value)
            restricted_factors.append(factor)
        factors = restricted_factors
    for var in bayes_net.variables():
        if var == var_query or var in varlist_evidence:
            continue
        relevant_factors = []
        remaining_factors = []
        for factor in factors:
            if var in factor.scope:
                relevant_factors.append(factor)
            else:
                remaining_factors.append(factor)
        if relevant_factors:
            combined = multiply_factors(relevant_factors)
            reduced = sum_out_variable(combined, var)
            remaining_factors.append(reduced)
        factors = remaining_factors
    final_factor = multiply_factors(factors)
    weights = []
    for value in var_query.domain():
        if var_query in varlist_evidence:
            weight = 0.0
            if value == var_query.get_evidence():
                weight = final_factor.get_value(())
        else:
            weight = final_factor.get_value((value,))
```

```
42          weights.append(weight)
43      return normalize(weights)
```

Line 1. Define variable elimination for a query variable conditioned on the listed evidence variables.

Line 2. Document the ordering of the answer and the requirement to preserve network factors.

Line 3. Check that the query variable belongs to this network.

Line 4. Reject a query about an unrelated variable.

Line 5. Get a new working list of factor references. The original factor objects are read only.

Line 6. Process each variable that the caller explicitly supplied as evidence.

Line 7. Check that each evidence variable belongs to this network.

Line 8. Reject evidence about an unrelated variable.

Line 9. Read the observed value previously stored by set_evidence.

Line 10. Check that the stored observation is valid; with the supplied domains this also catches unset evidence.

Line 11. Explain that the caller needs to set a valid observation first.

Line 12. Create a new working list for factors after applying this observation.

Line 13. Visit each factor in the current list.

Line 14. Only restrict factors that actually contain this evidence variable.

Line 15. Replace this local reference with a new restricted factor. The original factor is unchanged.

Line 16. Keep the restricted factor, or the original read-only reference if it did not contain this variable.

Line 17. Use the updated list for the next observation and for elimination.

Line 18. Consider eliminating variables in the network's supplied order.

Line 19. Keep the query variable and skip evidence variables, which were already removed by restriction.

Line 20. Skip the rest of this loop iteration.

Line 21. Prepare a list of factors that contain the variable currently being eliminated.

Line 22. Prepare a list of factors unaffected by this elimination.

Line 23. Visit each factor in the current list.

Line 24. Check whether the current factor depends on the variable being eliminated.

Line 25. Collect this factor for multiplication before summing out the variable.

Line 26. Handle the alternative to the preceding condition.

Line 27. Keep this unaffected factor for later steps.

Line 28. Perform an elimination only if at least one factor contains this variable.

Line 29. Multiply all factors that depend on this variable. Multiplication must happen before summing.

Line 30. Sum the selected variable out of that combined factor.

Line 31. Replace all the relevant factors with the single reduced factor.

Line 32. Update the working list for the next elimination.

Line 33. Multiply every remaining factor, including constants produced by restriction or summation.

Line 34. Prepare the unnormalized answer in the query domain's order.

Line 35. Visit each possible query value in the required output order.

Line 36. Handle the case where the caller has also observed the query variable itself.

Line 37. Give incompatible query values zero weight.

Line 38. Check whether this query value agrees with its observation.

Line 39. For the matching observed query value, use the remaining scalar evidence weight; all variables have been removed.

Line 40. Handle the alternative to the preceding condition.

Line 41. For an unobserved query, read its row from the final one-variable factor. The comma creates a one-element tuple.

Line 42. Append the weight for this query value without changing domain order.

Line 43. Divide by total evidence probability. Impossible evidence has total zero and raises ValueError.

Two details matter for correctness. First, do not sum a variable out of each factor separately: multiply all factors containing it before summing it out. In general, $\sum_x f(x)g(x)$ does not equal $(\sum_x f(x))(\sum_x g(x))$. Second, keep constant factors. An impossible observation can reduce a disconnected factor to zero. Dropping that factor would incorrectly turn impossible evidence into an apparently valid distribution.

If the query itself is observed, the answer is one at the observed value and zero elsewhere, provided the evidence has positive probability. The code still computes the evidence weight so that contradictory or impossible observations raise an error. Only variables explicitly listed in the evidence argument are treated as evidence. The solution reads their observations without changing or clearing them.

1.9 Tests and what they establish

The original factor tests check normalization, restriction, summation in both column positions, multiplication, and preservation of inputs. The original network test cell checks seven queries about the Alarm network. All pass. For example, querying Burglary without evidence returns $[0.001, 0.999]$, and querying MaryCalls given Burglary=True and Earthquake=False returns $[0.6586, 0.3414]$.

The additional test cell is part of the solution notebook. It performs these checks:

1. Normalize $[2, 6]$, verify that its input list is unchanged, and require an error for an empty list and for $[0, 0]$.

2. Multiply an empty list of factors, restrict a one-variable factor to a constant, and sum a one-variable factor into a constant.

3. Build a three-valued variable and a two-valued variable. Multiply tables whose scopes have opposite orders and check each product directly. Restrict and sum the three-valued variable, then check that the input scopes and tables are unchanged.

4. Independently compute the full joint probability for each assignment in the five-variable sample network. Each complete assignment contributes the product of the original network factors, without calling any of the five solution functions. Assignments that disagree with the evidence are skipped. Joint weights are accumulated by query value and divided by their total to obtain the reference distribution.

5. Compare VE with this reference for all five query variables and all 3^5 evidence patterns, where each variable is unobserved, True, or False. This produces $5 \times 3^5 = 1215$ checks, including observed queries. Check that evidence values remain unchanged after each call and that network factor scopes and values remain unchanged at the end.

6. Create a disconnected evidence variable whose observed value has probability zero. Require an error both when querying a different variable and when querying that evidence variable itself. Clear its observation and require an error when it is supplied as evidence without a value.

The tests use assertions, so a failed check stops execution. Floating-point comparisons use the supplied `close` helper to allow small rounding errors. The solution notebook contains the saved outputs of a successful sequential run. These checks are strong evidence for this implementation, not a mathematical proof for every possible network.

2 Problem 1: Hidden Markov models

2.1 Part (a): the joint probability

Weather is the hidden state Y_t, and the observed ice cream count is X_t. The three weather codes are 0 for cold, 1 for warm, and 2 for hot. The HMM assumptions give

$$P(y_1, \ldots, y_n, x_1, \ldots, x_n) = P(y_1)P(x_1 \mid y_1) \prod_{t=2}^{n} P(y_t \mid y_{t-1})P(x_t \mid y_t).$$

The initial state contributes once. Every day contributes one emission probability. Every day after the first contributes one transition probability. The notebook's existing written answer expresses this formula correctly.

2.2 The observed counts and probability tables

```
1  import math
2  outputs = [0, 0, 2, 2, 1] # X_1 .. X_N
3  # transition prob..row = today col = tomorrow
4  transition_prob = [[0.55, 0.25, 0.2],\
5                     [0.15, 0.7, 0.15],\
6                     [0.2, 0.4, 0.4]]
7  # how much ice cream eaten given the weather..row = weather col = ice cream eaten
8  emission_prob = [[0.1, 0.3, 0.6],\
9                   [0.2, 0.5, 0.3],\
10                  [0.6, 0.3, 0.1]]
11 start_prob = [0.3, 0.3, 0.4]
```

Line 1. Import Python's standard math module, used by the supplied approximate-equality tests.

Line 2. Record the observed ice cream counts for five days. These are observations, not temperatures.

Line 3. Comment explaining the two axes of the transition matrix.

Line 4. Start the transition matrix with the probabilities of tomorrow being cold, warm, or hot given that today is cold.

Line 5. Give the three probabilities for tomorrow given that today is warm.

Line 6. Give the three probabilities for tomorrow given that today is hot, and finish the matrix.

Line 7. Comment explaining the temperature rows and ice cream count columns of the emission matrix.

Line 8. Start the emission matrix: probabilities of eating zero, one, or two ice creams when cold.

Line 9. Give the ice cream count probabilities when warm.

Line 10. Give the ice cream count probabilities when hot, and finish the matrix.

Line 11. Give the initial probabilities for cold, warm, and hot on day one.

2.3 The HMM class and part (b)

```
1  import math
2
3  class HMM:
4    def __init__(self, transition_prob: list[list[float]], emission_prob: list[list[
       float]], start: list[float]):
5      # transition_prob[i][j] = P(Y_(t+1) = j | Y_t = i)  (row = current state, each
       row sums to 1)
6      self.transition_prob = transition_prob
7
8      # emission_prob[y][x] = P(X_t = x | Y_t = y)
9      self.emission_prob = emission_prob
10
11     # start[y] = P(Y_1 = y)
12     self.start = start
13
14   def joint_prob(self, outputs: list[int], hiddens: list[int]) -> float:
15     if len(outputs) != len(hiddens):
16         raise ValueError("outputs and hiddens must have the same length.")
17
18     if len(outputs) == 0:
19         return 1.0
20
21     probability = self.start[hiddens[0]]
22     probability *= self.emission_prob[hiddens[0]][outputs[0]]
23
24     for day in range(1, len(outputs)):
25         previous_weather = hiddens[day - 1]
26         current_weather = hiddens[day]
27         ice_cream_count = outputs[day]
28
29         probability *= self.transition_prob[previous_weather][current_weather]
30         probability *= self.emission_prob[current_weather][ice_cream_count]
31
32     return probability
```

Line 1. Import Python's standard math module, used by the supplied approximate-equality tests.

Line 3. Define the object that stores the model tables and computes joint probabilities.

Line 4. Initialize an HMM from its transition, emission, and initial distributions. Type hints describe expected types.

Line 5. Document that transition lookups go from the current state to the next state.

Line 6. Store the transition table on this model object.

Line 8. Document that the weather selects the row and ice cream count selects the column.

Line 9. Store the emission table on the model object.

Line 11. Document the meaning of an initial-state probability.

Line 12. Store the first day's weather distribution.

Line 14. Define the method that scores one proposed weather sequence together with the observed ice cream counts.

Line 15. Require one proposed weather state per observation.

Line 16. Reject mismatched sequences instead of computing an incomplete product.

Line 18. Handle the case where there are no days.

Line 19. Return the empty product, which is one; this also avoids accessing a nonexistent first day.

Line 21. Start with the probability of the first proposed weather state.

Line 22. Multiply by the likelihood of the first observation given the first weather state.

Line 24. Visit days after the first; Python indexes these from one rather than zero.

Line 25. Read the previous day's proposed weather state.

Line 26. Read the current day's proposed weather state.

Line 27. Read the current day's observed ice cream count.

Line 29. Multiply by the probability of this weather transition, in previous-to-current order.

Line 30. Multiply by the probability of today's observation given today's weather.

Line 32. Return the joint probability after every transition and emission has been included.

For observations `[0, 2, 1]` and proposed weather `[2, 0, 1]`, the calculation is

$$0.4 \times 0.6 \times 0.2 \times 0.6 \times 0.25 \times 0.5 = 0.0036.$$

These terms are, in order, the hot starting probability, the first emission, the hot-to-cold transition, the second emission, the cold-to-warm transition, and the third emission. No normalization is needed for part (b).

2.4 Constructing the model

```
1 hmm = HMM(transition_prob, emission_prob, start_prob)
```

Line 1. Construct the model using the three probability tables defined earlier.

2.5 The supplied joint-probability tests

```
1 # Test cases
2 assert math.isclose(hmm.joint_prob([0,0,2,2,1], [1,1,1,1,1]), 0.000129654)
3 assert math.isclose(hmm.joint_prob([0,0,2,2,1], [2,2,0,0,1]), 0.00028512)
4 print("joint_prob tests passed.")
```

Line 1. Label the supplied checks for the joint probability method.

Line 2. Check the joint probability for an all-warm proposed sequence, allowing normal floating-point rounding.

Line 3. Check a second proposed sequence that includes weather transitions.

Line 4. Print confirmation after both supplied assertions succeed.

2.6 Part (c): enumerate and rank every hidden sequence

```
1  import itertools
2
3  def get_most_likely_hidden_states(hmm: HMM, outputs: list[int]):
4      joint_probabilities = {}
5
6      for hiddens in itertools.product([0, 1, 2], repeat=len(outputs)):
7          joint_probabilities[hiddens] = hmm.joint_prob(outputs, hiddens)
8
9      total = sum(joint_probabilities.values())
10
11     if total == 0:
12         raise ValueError("The observed sequence has zero probability under this HMM."
       )
13
14     conditional_probabilities = {}
15
16     for hiddens, joint_probability in joint_probabilities.items():
17         conditional_probabilities[hiddens] = joint_probability / total
18
19     ranked_sequences = sorted(
20         conditional_probabilities,
21         key=conditional_probabilities.get,
22         reverse=True,
23     )
24
25     results = []
26
27     for hiddens in ranked_sequences[:10]:
28         results.append((hiddens, joint_probabilities[hiddens]))
29
30     return results
```

Line 1. Import the standard-library tools used to generate every possible weather sequence.

Line 3. Define exhaustive ranking of weather sequences for the supplied observations.

Line 4. Prepare a dictionary from weather sequence tuples to joint probabilities.

Line 6. Generate every length-n sequence over the three weather states. Each sequence is already a tuple.

Line 7. Score the proposed sequence with the method from part (b) and save its joint probability.

Line 9. Sum over all hidden sequences to obtain the marginal probability of the observed ice cream counts.

Line 11. Check whether the observations have zero probability under the model.

Line 12. Reject impossible observations because their conditional distribution is undefined.

Line 14. Prepare a separate dictionary for probabilities conditioned on the observations.

Line 16. Visit each proposed sequence and its saved joint probability.

Line 17. Apply Bayes' normalization: divide the joint probability by the observation probability.

Line 19. Begin sorting sequence keys by their conditional probabilities.

Line 20. Supply the dictionary, whose iteration yields the sequence keys.

Line 21. Use each sequence's stored conditional probability as its sorting score.

Line 22. Place larger scores first.

Line 23. Close the sorting call and assign the resulting ordered list.

Line 25. Prepare the final list of sequence-and-joint-probability pairs.

Line 27. Take at most the first ten ranked sequences.

Line 28. Append the sequence and its original joint probability, as requested by the question.

Line 30. Return the ranked list to the caller.

The denominator is

$$P(x_1, \ldots, x_n) = \sum_{y_1, \ldots, y_n} P(y_1, \ldots, y_n, x_1, \ldots, x_n).$$

Dividing by it gives a conditional probability for each proposed weather sequence. The denominator is the same for all sequences, so joint and conditional probabilities have the same ranking when the denominator is positive. The notebook still computes conditional probabilities explicitly, following the question, then returns joint probabilities in the final pairs, also following the question.

2.7 Running the five-day example

```
1 get_most_likely_hidden_states(hmm, outputs)
```

Line 1. Run the ranking on the five-day example; a notebook displays the returned list.

2.8 Verification and limits

The existing Problem 1 notebook was run in order without changing its file. Both supplied joint-probability tests pass. Independent checks confirm the three-day result above, the empty-sequence result, and that the returned ten five-day sequences exactly match the ten largest joint scores among all 243 possible weather sequences.

For n days, there are 3^n weather sequences. Computing each score takes $O(n)$ time, and sorting them takes $O(3^n \log(3^n))$ time. Overall time is $O(n3^n)$; storing all sequence tuples and scores also takes $O(n3^n)$ space. This straightforward method is appropriate for the short assignment example. Problem 2 can also require large intermediate factors: multiplying factors enumerates every joint assignment in their combined scope. No faster elimination order or specialized inference library is used.

Both solutions assume valid model tables and finite domains. Problem 1 assumes valid state and observation indices. Very long probability products can underflow to zero with floating-point arithmetic; log-space computation is outside this short-example implementation. Neither solution substitutes epsilon for a zero normalizing constant because that would change the probability calculation.